E-Commerce Backend Project

Phase 1: Core Shop (Catalog, Cart and Orders)

Project Type: Full-stack e-commerce application (backend first, React / JavaScript frontend later)
Technology Stack: Java 17+, Spring Boot 3.x, Spring Data JPA, Spring Security (JWT), H2 Database (development), MySQL / PostgreSQL (later)
Focus Areas: RESTful APIs, Database Relationships (1:1, 1:N, N:1), JPA Annotations, JWT Authentication, DTOs and Validation

Assignment: E-Commerce Backend API

Build a RESTful backend for an online store using Java and Spring Boot. Phase 1 delivers a working shop: users can register and log in, browse products by category, manage their addresses, add items to a cart and place orders. Admins manage the catalog and update order statuses.

The frontend (React or plain JavaScript) will be added later, so the API must be frontend-ready: JSON responses built from DTOs (never raw entities), CORS enabled, consistent error responses and documented endpoints.

Entity Relationship Overview

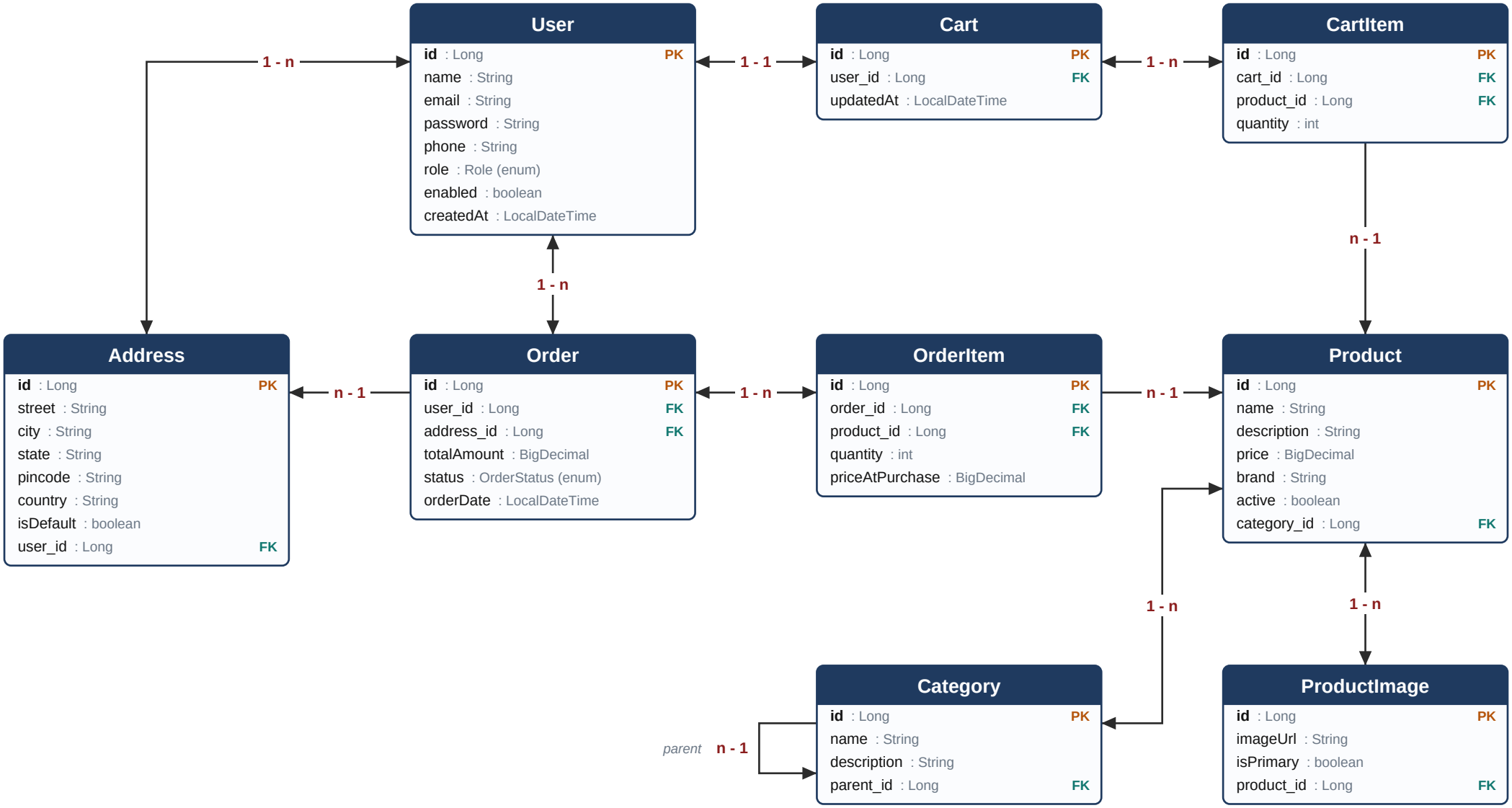
Your database should implement the following relationships. The full diagram with every field is on the next page (Figure 1).

```
User (1) <----> (*) Address
User (1) <----> (1) Cart
User (1) <----> (*) Order

Cart (1) <----> (*) CartItem
CartItem (*) ----> (1) Product

Order (1) <----> (*) OrderItem
OrderItem (*) ----> (1) Product
Order (*) ----> (1) Address                  [shipping address]

Category (1) <----> (*) Product
Category (*) ----> (1) Category              [parent, self-reference]

Product (1) <----> (*) ProductImage
```

Legend: `---->` unidirectional (only the source entity holds the reference), `<---->` bidirectional (both entities hold a reference to each other).

Enums (not separate tables): `Role` = CUSTOMER, SELLER, ADMIN (field on User) and `OrderStatus` = PENDING, PAID, SHIPPED, DELIVERED, CANCELLED (field on Order). Store both with `@Enumerated(EnumType.STRING)`.

Figure 1: Phase 1 ER Diagram



Legend → Unidirectional (source holds the reference) ↔ Bidirectional (both hold a reference) **PK** Primary key **FK** Foreign key

Key Points

- Use `@OneToMany` and `@ManyToOne` (bidirectional, with `mappedBy`) for User-Address, Cart-CartItem, Order-OrderItem, Category-Product and Product-ProductImage
- Use `@OneToOne` with `mappedBy` for User-Cart (the Cart table owns the `user_id` foreign key)
- Use unidirectional `@ManyToOne` for CartItem-Product, OrderItem-Product and Order-Address (the other side never needs to navigate back)
- Model the category tree with a self-referencing `@ManyToOne` on `Category.parent`
- Use `CascadeType.ALL` with `orphanRemoval = true` for CartItem, OrderItem and ProductImage so children are removed with their parent. Never cascade REMOVE from an item to its Product
- Prefer `FetchType.LAZY` and return DTOs; if you must return entities, use `@JsonIgnore` or `@JsonManagedReference` / `@JsonBackReference` to prevent circular references in JSON
- Use `BigDecimal` for every price or amount, never `double`
- Copy the product price into `OrderItem.priceAtPurchase` when the order is placed, so later price changes never alter past orders

Requirements

Core Features (Must Have)

1. Register and Login (JWT)

- Fields: name (required), email (required, unique, valid format), password (min 8 characters, stored with BCrypt), phone (optional)
- New users get the role CUSTOMER by default; admins are seeded or created manually
- Login returns a JWT; protected endpoints require `Authorization: Bearer <token>`

2. Manage Categories (Admin)

- Fields: name (required), description (optional), parent (optional, for subcategories)
- Create, update, delete and list categories; deleting a category that still has products must return 409 Conflict

3. Manage Products (Admin)

- Fields: name (required), description, price (required, greater than 0), brand, active, category
- Create and update products; deleting a product sets `active = false` (soft delete) so old orders stay valid
- Add and remove product images (an image URL and an isPrimary flag)

4. Browse Products (Public)

- Get all products with pagination (`page`, `size`, `sort`); only active products are returned
- Filter by category, keyword (name or brand) and price range using query parameters
- Get a single product by ID and handle the not-found case with 404

5. Manage Addresses

- A logged-in user can create, list, update and delete their own addresses and mark one as default
- A user must never be able to read or change another user's address

6. Shopping Cart

- Every user has one cart; view the cart with line subtotals and a total
- Add a product (adds to quantity if it is already in the cart), change quantity, remove an item and clear the cart
- Reject quantity less than 1 and products that are inactive

7. Place an Order

- Convert the current cart into an order using a selected shipping address
- Copy each cart line into an OrderItem with `priceAtPurchase`, calculate `totalAmount`, set the status to PENDING and clear the cart
- Return 400 if the cart is empty or the address does not belong to the user

8. View and Manage Orders

- Customers can list their own orders and view one order by ID; they can cancel an order only while it is PENDING
- Admins can list all orders and move an order through PAID, SHIPPED and DELIVERED

Technical Requirements

- Use Java 17+ and Spring Boot 3.x with Maven or Gradle
- Use Spring Data JPA for all database operations and an SQL database (H2 during development, MySQL or PostgreSQL later)
- Use Spring Security with stateless JWT authentication and role-based access (CUSTOMER, SELLER, ADMIN)
- Use request and response DTOs with Bean Validation (`@NotBlank`, `@Email`, `@Positive`, ...)
- Implement proper HTTP status codes (200, 201, 204, 400, 401, 403, 404, 409)
- Add a global exception handler with `@RestControllerAdvice` that returns meaningful error messages
- Enable CORS so a React or JavaScript frontend can call the API
- Follow a layered structure: `config, controller, service, repository, entity, dto, exception, security`
- Write clean, readable code with proper naming conventions

Database Relationships (Important)

Implement the following JPA relationships to demonstrate your understanding of database modeling.

1. One-to-Many: User to Addresses

- **User** fields: id, name, email, password, phone, role (enum), enabled, createdAt
- **Address** fields: id, street, city, state, pincode, country, isDefault, user
- Each address belongs to one user; a user can have many addresses
- Endpoints: `POST /api/addresses`, `GET /api/addresses` (current user), `PUT /api/addresses/{id}`, `DELETE /api/addresses/{id}`

2. One-to-One and One-to-Many: User to Cart to CartItems

- **Cart** fields: id, user, updatedAt (one cart per user, use `@OneToOne`)
- **CartItem** fields: id, cart, product, quantity (many items per cart, each pointing to one product)
- Add a unique constraint on (cart_id, product_id) so a product appears once per cart
- Endpoints: `GET /api/cart`, `POST /api/cart/items`, `PUT /api/cart/items/{itemId}`, `DELETE /api/cart/items/{itemId}`, `DELETE /api/cart`

3. One-to-Many: User to Orders, Order to OrderItems

- **Order** fields: id, user, address (shipping), totalAmount, status (enum), orderDate
- **OrderItem** fields: id, order, product, quantity, priceAtPurchase
- Each order belongs to one user and one shipping address and contains many order items
- Use `CascadeType.ALL` from Order to OrderItem; deleting a Product must never delete past order items

- Endpoints: `POST /api/orders`, `GET /api/orders`, `GET /api/orders/{id}`, `PUT /api/orders/{id}/cancel`, `PUT /api/admin/orders/{id}/status` (admin)

4. One-to-Many: Category to Products (and Category self-reference)

- **Category** fields: id, name, description, parent (nullable, points to another Category)
- **Product** fields: id, name, description, price, brand, active, category
- Each product belongs to one category; a category can have many products and many subcategories
- Endpoints: `POST /api/categories`, `GET /api/categories`, `GET /api/categories/{id}/products`

5. One-to-Many: Product to ProductImages

- **ProductImage** fields: id, imageUrl, isPrimary, product
- Each product can have many images; at most one image is marked primary
- Endpoints: `POST /api/products/{id}/images`, `DELETE /api/products/{id}/images/{imageId}`

Data Dictionary

Entity	Fields	References
User	id (PK), name, email (unique), password, phone, role (enum), enabled, createdAt	-
Address	id (PK), street, city, state, pincode, country, isDefault, user_id (FK)	User (N:1)
Category	id (PK), name, description, parent_id (FK)	Category (N:1, parent)
Product	id (PK), name, description, price, brand, active, category_id (FK)	Category (N:1)
ProductImage	id (PK), imageUrl, isPrimary, product_id (FK)	Product (N:1)
Cart	id (PK), user_id (FK, unique), updatedAt	User (1:1)
CartItem	id (PK), cart_id (FK), product_id (FK), quantity	Cart (N:1), Product (N:1)
Order	id (PK), user_id (FK), address_id (FK), totalAmount, status (enum), orderDate	User (N:1), Address (N:1)
OrderItem	id (PK), order_id (FK), product_id (FK), quantity, priceAtPurchase	Order (N:1), Product (N:1)

Additional Bonus Points

- Add unit tests for the service layer (JUnit 5 and Mockito)
- Add API documentation with Swagger / OpenAPI (springdoc)
- Use Flyway for database migrations once the schema is stable
- Check product stock before adding to cart or placing an order (preview of Phase 2)
- Prevent orphan cart items and order items when parents are deleted
- Add a Dockerfile and a docker-compose file for the app and database

Roadmap After Phase 1

Phase	New Entities	What it adds
Phase 2	Payment, Shipment, Inventory, OrderStatusHistory	Payment gateway (Razorpay or Stripe), shipping and tracking, stock handling, order status log
Phase 3	Review, Wishlist, WishlistItem, Coupon, ProductVariant, Notification, Brand	Ratings and reviews, wishlist, discount codes, size / color variants, notifications, brand as its own entity

Deliverables

1. Source Code

- Push your code to a GitHub repository (public or private) and commit regularly with meaningful messages

2. README.md should include:

- Instructions to run the application
- API endpoints documentation (or a Postman collection)
- Database schema explanation: describe the relationships between entities
- Any assumptions you made
- Technologies / dependencies used

3. Sample API Requests (in README or Postman collection)

- Include examples for registering, logging in, creating products, adding to cart and placing an order
- Show an example of filtering products by category and price range

Sample API Endpoints

Auth

Method	Endpoint	Description	Access
`POST`	`/api/auth/register`	Register a new user	Public
`POST`	`/api/auth/login`	Login and receive a JWT	Public

Addresses

Method	Endpoint	Description	Access
`POST`	`/api/addresses`	Add an address	Customer
`GET`	`/api/addresses`	Get my addresses	Customer
`PUT`	`/api/addresses/{id}`	Update an address	Customer
`DELETE`	`/api/addresses/{id}`	Delete an address	Customer

Categories

Method	Endpoint	Description	Access
`POST`	`/api/categories`	Create a category	Admin
`GET`	`/api/categories`	Get all categories	Public
`GET`	`/api/categories/{id}/products`	Get products in a category	Public
`PUT`	`/api/categories/{id}`	Update a category	Admin
`DELETE`	`/api/categories/{id}`	Delete a category	Admin

Products

Method	Endpoint	Description	Access
`POST`	`/api/products`	Create a product	Admin
`GET`	`/api/products`	List products (?category=&keyword=&minPrice=&maxPrice=&page=&size=)	Public

Method	Endpoint	Description	Access
GET	/api/products/{id}	Get product by ID	Public
PUT	/api/products/{id}	Update a product	Admin
DELETE	/api/products/{id}	Deactivate a product (soft delete)	Admin
POST	/api/products/{id}/images	Add an image to a product	Admin
DELETE	/api/products/{id}/images/{imageId}	Remove a product image	Admin

Cart

Method	Endpoint	Description	Access
GET	/api/cart	View my cart	Customer
POST	/api/cart/items	Add a product to the cart	Customer
PUT	/api/cart/items/{itemId}	Change item quantity	Customer
DELETE	/api/cart/items/{itemId}	Remove an item	Customer
DELETE	/api/cart	Clear the cart	Customer

Orders

Method	Endpoint	Description	Access
POST	/api/orders	Place an order from the cart	Customer
GET	/api/orders	Get my orders	Customer
GET	/api/orders/{id}	Get order by ID	Customer
PUT	/api/orders/{id}/cancel	Cancel a PENDING order	Customer
GET	/api/admin/orders	Get all orders	Admin
PUT	/api/admin/orders/{id}/status	Update order status	Admin

Sample Request/Response Examples

1. Register a User

`POST /api/auth/register`

Request:

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "password": "Secret@123",
  "phone": "9876543210"
}
```

Response (201 Created):

```
{
  "id": 1,
  "name": "John Doe",
  "email": "john@example.com",
  "role": "CUSTOMER"
}
```

2. Login

`POST /api/auth/login`

Request:

```
{
  "email": "john@example.com",
  "password": "Secret@123"
}
```

Response (200 OK):

```
{
  "token": "eyJhbGciOiJIUzI1NiJ9...",
  "tokenType": "Bearer",
  "role": "CUSTOMER"
}
```

3. Create a Product (Admin)

`POST /api/products`

Request:

```
{
  "name": "Wireless Mouse",
  "description": "2.4 GHz wireless optical mouse",
  "price": 799.00,
  "brand": "Logitech",
  "categoryId": 3
}
```

Response (201 Created):

```
{
  "id": 12,
  "name": "Wireless Mouse",
  "description": "2.4 GHz wireless optical mouse",
  "price": 799.00,
  "brand": "Logitech",
  "active": true,
  "category": { "id": 3, "name": "Electronics" },
  "images": []
}
```

4. Add a Product to the Cart

`POST /api/cart/items`

Request:

```
{
  "productId": 12,
  "quantity": 2
}
```

Response (200 OK):

```
{
  "id": 1,
  "items": [
    {
      "id": 5,
      "product": { "id": 12, "name": "Wireless Mouse", "price": 799.00 },
      "quantity": 2,
      "subtotal": 1598.00
    }
  ],
  "totalAmount": 1598.00
}
```

5. Place an Order

`POST /api/orders`

Request:

```
{
  "addressId": 4
}
```

Response (201 Created):

```
{
  "id": 101,
  "status": "PENDING",
  "shippingAddress": {
    "id": 4,
    "street": "12 MG Road",
    "city": "Bengaluru",
    "state": "Karnataka",
    "pincode": "560001"
  },
  "items": [
    {
      "productId": 12,
      "productName": "Wireless Mouse",
      "quantity": 2,
      "priceAtPurchase": 799.00
    }
  ],
  "totalAmount": 1598.00,
  "orderDate": "2026-10-25T14:30:00"
}
```

6. Filter Products

`GET /api/products?category=3&minPrice=500&maxPrice=1000&page=0&size=10`

Response (200 OK):

```
{
  "content": [
    { "id": 12, "name": "Wireless Mouse", "price": 799.00, "brand": "Logitech" }
  ],
  "page": 0,
  "size": 10,
  "totalElements": 1,
  "totalPages": 1
}
```

7. Error Response

`GET /api/products/999`

Response (404 Not Found):

```
{
  "timestamp": "2026-10-25T14:35:10",
  "status": 404,
  "error": "Not Found",
  "message": "Product with id 999 not found",
  "path": "/api/products/999"
}
```

Submission Guidelines

Keep the project in a Git repository and work phase by phase:

- Finish Phase 1 so the app runs end to end (register, browse, cart, order) before starting Phase 2
- Push to GitHub after each module and share the repository link
- Add a brief note about your approach (2-3 sentences) at the top of the README

Happy coding!